Effectiveness of chlorine on the microbial reduction in agricultural water

Naim Montazeri, Simon Riley, and Arie Havelaar

Table of contents

1. Preliminary setup

Run this script from within the R project - the code now uses relative paths which may not work otherwise.

Load Required Packages

```
# Load packages
library(here)
library(writexl)
library(tidyverse)
library(ggpubr)
library(nlme)
library(ggResidpanel)
library(performance)
library(emmeans)
```

```r
library(marginaleffects)
library(multcomp)
library(multcompView)

sessioninfo::session_info(pkgs = c("attached"))
```

```
─ Session info ─────────────────────────────────────────────────────────────
 setting  value
 version  R version 4.4.2 (2024-10-31)
 os       macOS Sequoia 15.3.2
 system   x86_64, darwin20
 ui       X11
 language (EN)
 collate  en_US.UTF-8
 ctype    en_US.UTF-8
 tz       America/New_York
 date     2025-04-19
 pandoc   3.2 @
/Applications/RStudio.app/Contents/Resources/app/quarto/bin/tools/x86_64/
(via rmarkdown)
 quarto   1.3.433 @ /usr/local/bin/quarto

─ Packages ─────────────────────────────────────────────────────────────────
 package         * version date (UTC) lib source
 dplyr           * 1.1.4   2023-11-17 [1] CRAN (R 4.4.0)
 emmeans         * 1.10.7  2025-01-31 [1] CRAN (R 4.4.1)
 forcats         * 1.0.0   2023-01-29 [1] CRAN (R 4.4.0)
 ggplot2         * 3.5.1   2024-04-23 [1] CRAN (R 4.4.0)
 ggpubr          * 0.6.0   2023-02-10 [1] CRAN (R 4.4.0)
 ggResidpanel    * 0.3.0   2019-05-31 [1] CRAN (R 4.4.0)
 here            * 1.0.1   2020-12-13 [1] CRAN (R 4.4.0)
 lubridate       * 1.9.4   2024-12-08 [1] CRAN (R 4.4.1)
 marginaleffects * 0.25.0  2025-02-01 [1] CRAN (R 4.4.1)
 MASS            * 7.3-65  2025-02-28 [1] CRAN (R 4.4.1)
 multcomp        * 1.4-28  2025-01-29 [1] CRAN (R 4.4.1)
 multcompView    * 0.1-10  2024-03-08 [1] CRAN (R 4.4.0)
 mvtnorm         * 1.3-3   2025-01-10 [1] CRAN (R 4.4.1)
 nlme            * 3.1-167 2025-01-27 [1] CRAN (R 4.4.1)
 performance     * 0.13.0  2025-01-15 [1] CRAN (R 4.4.1)
 purrr           * 1.0.4   2025-02-05 [1] CRAN (R 4.4.1)
 readr           * 2.1.5   2024-01-10 [1] CRAN (R 4.4.0)
 stringr         * 1.5.1   2023-11-14 [1] CRAN (R 4.4.0)
 survival        * 3.8-3   2024-12-17 [1] CRAN (R 4.4.1)
 TH.data         * 1.1-3   2025-01-17 [1] CRAN (R 4.4.1)
 tibble          * 3.2.1   2023-03-20 [1] CRAN (R 4.4.0)
 tidyr           * 1.3.1   2024-01-24 [1] CRAN (R 4.4.0)
 tidyverse       * 2.0.0   2023-02-22 [1] CRAN (R 4.4.0)
 writexl         * 1.5.1   2024-10-04 [1] CRAN (R 4.4.1)
```

```
 [1] /Library/Frameworks/R.framework/Versions/4.4-x86_64/Resources/library
 * ── Packages attached to the search path.
```

_

Set global options

```
options(contrasts = c('contr.sum', 'contr.poly'),
        dplyr.width = Inf, pillar.print_max = 50, pillar.print_min = 50)

knitr::opts_chunk$set(eval = T, echo = T, cache = T, fig.align = 'center')

set.seed(1484) #accroding to https://www.random.org
```

Define custom functions

The candidate models are Weibull and log-logistic. The derivation of the reparameterized log-logistic is provided at the end of this document.

```
# Weibull and reparameterized weibull
 wb = function (x, log10n0, alpha, beta) {
   log10n0 - (x/exp(alpha))^beta
 }

 wbrepar = function(x, log10n0, TD, beta, D = 1){
  log10n0 - D*(x/TD)^beta
}

# Log-logistic and reparameterized log-logistic
lg = function(x, log10n0, kappa, sigma){
   log10n0 - log10(1+ exp((log(x)-kappa)/sigma^2))
 }

lgrepar = function(x, log10n0, TD, sigma, D = 1){
  log10n0 - log10(1+ exp((log(x)- (log(TD) - log(10^D -
1)*sigma^2))/sigma^2))
}
```

2. Chlorine demand (CLD)

2.1 Data entry

```
chl = read.delim(here('study 1', 'Data (study 1)', "obj1_chl_NM.txt")) |>
  mutate(across(1:2, factor),
         sample = fct_relevel(sample, 'di', 'ag1', 'ag2'))
str(chl)
```

```
'data.frame':   21 obs. of  7 variables:
 $ sanitizer       : Factor w/ 1 level "accutab": 1 1 1 1 1 1 1 1 1 1 ...
 $ sample          : Factor w/ 3 levels "di","ag1","ag2": 2 2 2 2 2 2 2 3 3 3
...
 $ chl.added       : int  2 4 10 15 20 30 40 2 4 10 ...
 $ chl.measured.ave: num  0.85 2.87 8.23 11.77 16.4 ...
 $ chl.measured.se : num  0.11 0.19 0.36 0.58 0.41 0.35 0.65 0.05 0.07 0.4
...
 $ CLD.ave         : num  1.15 1.13 1.77 3.23 3.6 4.35 4.15 1.73 3.68 6.42
...
 $ CLD.se          : num  0.11 0.19 0.36 0.58 0.41 0.35 0.65 0.05 0.07 0.4
...
```

```
saveRDS(chl, here('study 1', 'Results (study 1)', 'CLD_all_ave.RDS'))

# Grouping based on sanitizer for each Ag water
CLD_m1 = readRDS(here('study 1', 'Results (study 1)', "CLD_all_ave.RDS")) |>
  dplyr::filter(sample=="ag1" | sample=="ag2" |
                  sample=="di" &  sanitizer=="accutab")
```

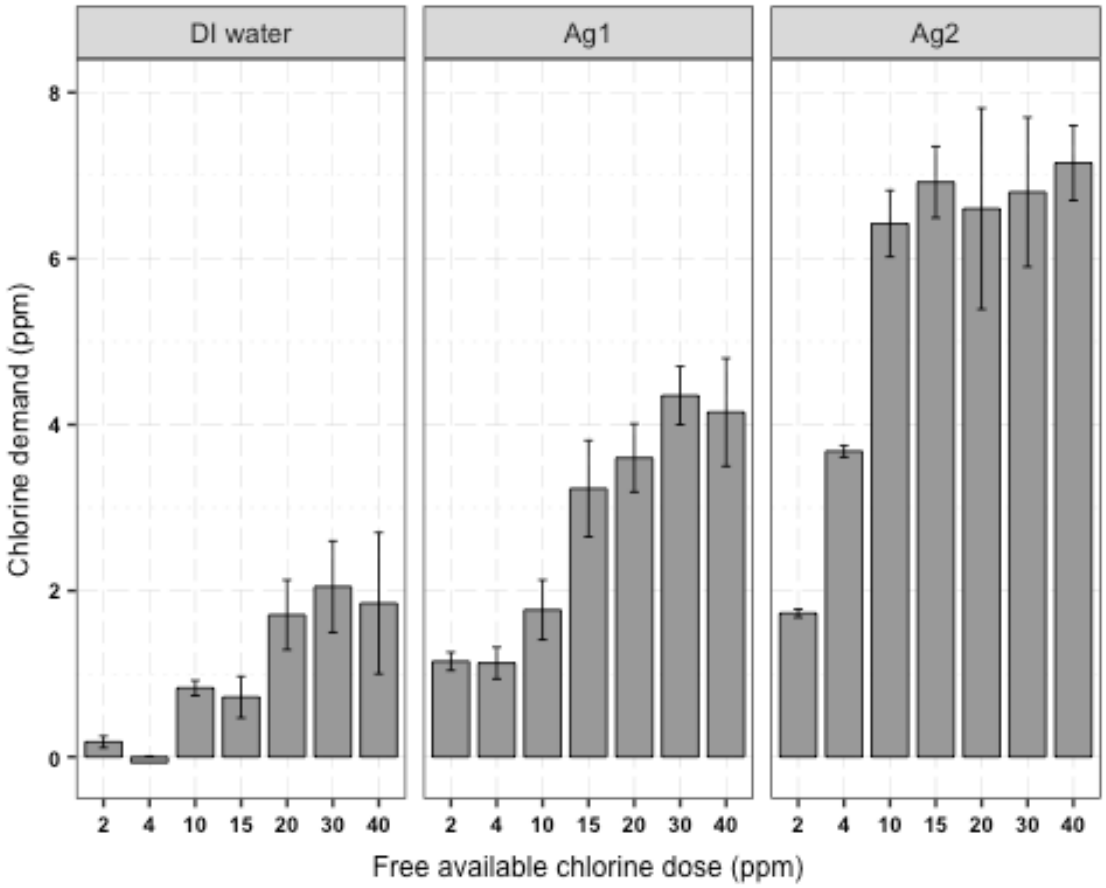
2.2 Plot CLD

```
CLD_m1 =  readRDS(here('study 1', 'Results (study 1)', "CLD_all_ave.RDS")) |>
  dplyr::filter(sample=="ag1" | sample=="ag2" |
                  sample=="di" &  sanitizer=="accutab") |>
  mutate(chl.added = factor(chl.added))

str(CLD_m1)
```

```
'data.frame':   21 obs. of  7 variables:
 $ sanitizer       : Factor w/ 1 level "accutab": 1 1 1 1 1 1 1 1 1 1 ...
 $ sample          : Factor w/ 3 levels "di","ag1","ag2": 2 2 2 2 2 2 2 3 3 3
...
 $ chl.added       : Factor w/ 7 levels "2","4","10","15",..: 1 2 3 4 5 6 7 1
2 3 ...
 $ chl.measured.ave: num  0.85 2.87 8.23 11.77 16.4 ...
 $ chl.measured.se : num  0.11 0.19 0.36 0.58 0.41 0.35 0.65 0.05 0.07 0.4
...
 $ CLD.ave         : num  1.15 1.13 1.77 3.23 3.6 4.35 4.15 1.73 3.68 6.42
...
 $ CLD.se          : num  0.11 0.19 0.36 0.58 0.41 0.35 0.65 0.05 0.07 0.4
...
```

```
var_names = c(
 "sample" = "Samples",
 "ag1"="Ag1",
 "ag2" = "Ag2",
 "di" = "DI water"
  )

p_chl_demand = CLD_m1 |>
```

```
  filter (sample =="di"|sample=="ag1" | sample=="ag2") |>
  ggplot (aes(x=chl.added, y=CLD.ave, fill=sample)) +
  geom_bar (position = position_dodge(0.8),
            width=0.8,
            color ="black",
            stat="identity",
            linewidth=0.3) +
  facet_grid (~ sample, labeller = as_labeller(var_names)) +
  geom_errorbar(aes(ymin=CLD.ave-CLD.se,
                    max=CLD.ave+CLD.se),
                position = position_dodge(0.8),
                linewidth=0.3,
                width=0.2) +
  labs (
    x = "Free available chlorine dose (ppm)",
    y = "Chlorine demand (ppm)",
    size = 9, face="bold") +
  scale_fill_manual(values=c("grey60",  "grey60", "grey60"),
                    name="",
                    breaks=c("di", "ag1", "ag2"),
                    labels=c("DI water", "Ag1", "Ag2")) +
  ylim (-0.1, 8) +
  theme_bw (base_family="") +
  theme(
    text = element_text(family = "Arial"),
    legend.position = "none",
    plot.title = element_text(color="black", size=9, face = "bold", hjust =
0.5),
    axis.text.x=element_text(size = 7, color="black", face="bold", hjust =
0.5, vjust = 0.5),
    axis.text.y=element_text (size = 7, color="black", face="bold"),
    axis.title.x = element_text (size = 9, vjust = -1),
    axis.title.y = element_text (size = 9, vjust = 2),
    panel.grid.major = element_line (color="gray", linewidth=0.1,
linetype="longdash"),
    panel.grid.minor = element_line (color="lightgray", linewidth=0.1,
linetype="dotted"),
    panel.background = element_rect (fill =  "transparent"),
    legend.background = element_rect (fill = "transparent", color =
"transparent"),
    plot.background = element_rect (fill = "transparent", color =
"transparent")) +
  guides(
    color = guide_legend(override.aes = list(size = 5))
  )

p_chl_demand
```




```
ggsave(file=here('study 1', 'Results (study 1)', "Fig_1 CLD_plot.tiff"),
       plot = p_chl_demand,
       width=9, height=3.9, units = c("cm"), dpi=600)
```

3. Control group

3.1 Data entry and basic calculations

```
m1c = readRDS(here('study 1', 'Data (study 1)', 'obj1_m1.rda')) |>
 filter(sample == "di" & time == 5) |>
 mutate(across(c(1, 4, 7:8, 10), factor))

m1c_w = m1c |>
 tidyr::spread (rep, survival)

# calculate mean, sd, and se
m1c_w$mean = apply(
 m1c_w[,c(11:12)], 1, base::mean, na.rm=TRUE) # calculate mean
m1c_w$sd = apply(
  m1c_w[,c(11:12)], 1, stats::sd, na.rm=TRUE) # calculate Std Dev
m1c_w$se = m1c_w$sd/sqrt(3) # calculate Std Error
```

```
m1c_w |>
 dplyr::select (sanitizer, sample, chl.measured, temp, time, microb, assay,
mean, sd, se) |>
 saveRDS(here('study 1', 'Results (study 1)', "m1c_w.RDS"))

# Log10 reduction for E coli 0 ppm to 2 ppm
ec_chl0 <- m1c |>
  filter(microb == "ec" & chl.added == 0 & time=="5") |>
  dplyr::select(survival)
mean(ec_chl0$survival)

[1] 6.668333

sd(ec_chl0$survival)/sqrt(3)

[1] 0.04733685

ec_chl2 <- m1c |>
  filter(microb == "ec" & chl.added == 2 & time=="5") |>
  dplyr::select(survival)
mean(ec_chl2$survival)

[1] 1.933

sd(ec_chl2$survival)/sqrt(3)

[1] 0.2187792

mean(ec_chl0$survival-ec_chl2$survival)

[1] 4.735333

sd(ec_chl0$survival-ec_chl2$survival)/sqrt(3)

[1] 0.2651517

# Log reduction for TV 0 ppm to 4 ppm
tv_chl0 <- m1c |>
  filter(microb == "tv" & chl.added == 0 & time=="5") |>
  dplyr::select(survival)
mean(tv_chl0$survival)

[1] 5.205

sd(tv_chl0$survival)/sqrt(3)

[1] 0.06236452

tv_chl2 <- m1c |>
  filter(microb == "tv" & chl.added == 2 & time=="5") |>
  dplyr::select(survival)
mean(tv_chl2$survival)
```

```
[1] 2.535333
```

```
sd(tv_chl2$survival)/sqrt(3)
```

```
[1] 0.2770754
```

```
mean(tv_chl0$survival-tv_chl2$survival)
```

```
[1] 2.669667
```

```
sd(tv_chl0$survival-tv_chl2$survival)/sqrt(3)
```

```
[1] 0.316151
```

```
tv_chl4 <- m1c |>
  filter(microb == "tv" & chl.added == 4 & time=="5") |>
  dplyr::select(survival)
mean(tv_chl4$survival)
```

```
[1] 1.655
```

```
sd(tv_chl4$survival)/sqrt(3)
```

```
[1] 0.1924864
```

```
mean(tv_chl0$survival-tv_chl4$survival)
```

```
[1] 3.55
```

```
sd(tv_chl0$survival-tv_chl4$survival)/sqrt(3)
```

```
[1] 0.248037
```

```
# Max inactivation
mean(ec_chl0$survival) - 1
```

```
[1] 5.668333
```

```
mean(tv_chl0$survival) - 0.347
```

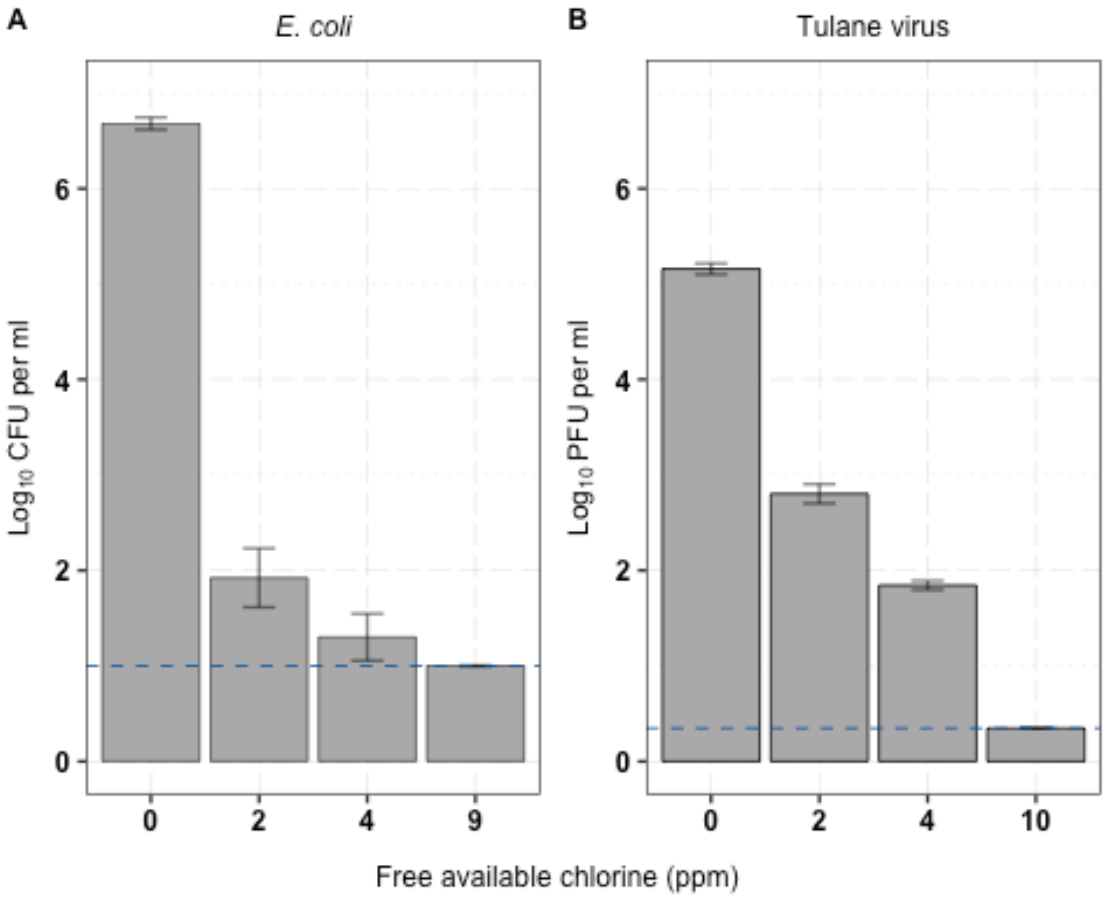
```
[1] 4.858
```

3.2 Plot control group

```
m1c_w = readRDS(here('study 1', 'Results (study 1)', "m1c_w.RDS")) |>
 filter(time=="5")
m1c_w$chl.measured.fac = as.factor (round(m1c_w$chl.measured, 0))

# LOD for each microbe
lod_tv = 0.347
lod_ec = 1.0

p_tv = m1c_w |>
 filter(microb=="tv") |>
 ggplot (aes(x=chl.measured.fac, y=mean)) +
```

```
 geom_bar(stat = "identity", position = position_dodge(0.1),
          linewidth = 0.3, fill = "darkgray", color = 'black') +
 geom_errorbar (aes(x=chl.measured.fac, ymin=mean-se, ymax=mean+se),
                 width=0.3, alpha=0.5, color = "black",
                 position = position_dodge(0.1)) +
 geom_hline (yintercept=lod_tv, linetype="dashed", color = "#00529b",
linewidth = 0.3) +
 scale_shape_manual(values=c (3, 6)) +
 ggtitle (expression(paste("Tulane virus"))) +
 labs (y= bquote (Log[10]~PFU~per~ml)) +
 ylim (0, 7) +
  theme_bw (base_family="") +
  theme(
    plot.title = element_text(color="black", size=9, face = "bold", hjust =
0.5),
    axis.text.x=element_text (size = 9, color="black", face="bold"),
    axis.text.y=element_text (size = 9, color="black", face="bold"),
    axis.title.x = element_blank(),
    axis.title.y = element_text (size = 9, vjust = 2),
    panel.grid.major = element_line (color="gray", linewidth=0.1,
linetype="longdash"),
    panel.grid.minor = element_line (color="lightgray", linewidth=0.1,
linetype="dotted"),
    panel.background = element_rect (fill =  "transparent"),
    legend.background = element_rect (fill = "transparent", color =
"transparent"),
    plot.background = element_rect (fill = "transparent", color =
"transparent"))

 p_ec = m1c_w |>
 filter(microb=="ec" & chl.measured!="14.3") |>
 ggplot (aes(x=chl.measured.fac, y=mean)) +
 geom_bar(stat = "identity", position = position_dodge(0.1),
          linewidth = 0.2, fill = "darkgray", color = 'black') +
 geom_errorbar (aes(x=chl.measured.fac, ymin=mean-se, ymax=mean+se),
                 width=0.3, alpha=0.5, color = "black",
                 position = position_dodge(0.1)) +
 geom_hline (yintercept=lod_ec, linetype="dashed", color = "#00529b",
linewidth = 0.3) +
 scale_shape_manual(values=c (3, 6)) +
 ggtitle (expression(italic("E. coli"))) +
 labs (y= bquote (Log[10]~CFU~per~ml)) +
 ylim (0, 7) +
  theme_bw (base_family="") +
  theme(
    plot.title = element_text(color="black", size=9, face = "bold", hjust =
0.5),
    axis.text.x=element_text (size = 9, color="black", face="bold"),
    axis.text.y=element_text (size = 9, color="black", face="bold"),
    axis.title.x = element_blank(),
```

```
    axis.title.y = element_text (size = 9, vjust = 2),
    panel.grid.major = element_line (color="gray", linewidth=0.1,
linetype="longdash"),
    panel.grid.minor = element_line (color="lightgray", linewidth=0.1,
linetype="dotted"),
    panel.background = element_rect (fill =  "transparent"),
    legend.background = element_rect (fill = "transparent", color =
"transparent"),
    plot.background = element_rect (fill = "transparent", color =
"transparent"))

combined_plot = ggpubr::ggarrange(p_ec,
                                  p_tv, ncol = 2,
                                   labels = c("A", "B"),
                                   font.label = list(size=9, face="bold",
family="Arial"
                                   ))
combined_plot <- annotate_figure(combined_plot,
                                 bottom = text_grob("Free available chlorine
(ppm)",
                                                    size = 9,
                                                    family = "Arial"))

combined_plot
```




```
ggsave(plot=combined_plot,
       file= here('study 1', 'Results (study 1)',
"Fig_2_control_group_plot.tiff"),
       width=9, height=5, units = c("cm"), dpi=600)
```

4. Modeling microbial inactivation kinetics

4.1 Data entry and approach

```
m1_file = here('study 1', 'Data (study 1)', 'obj1_m1_NM.txt')
m1 = read.table(file = m1_file, sep = '\t', header = T) |>
  mutate(across(c(1, 4, 7:8, 10), factor),
         temp_fac = factor(temp),
         time_fac = factor(time, labels = c('5min', '10min')),
         microb = fct_recode(microb, 'ec' = 'ecoli', 'tv' = 'tv'))

saveRDS(m1, here('study 1', 'Data (study 1)', 'obj1_m1.rda'))
```

Since there were insufficient time points included in the study to fit a primary model, time was initially included as a (categorical) fixed effect for the model parameters in the Weibull and log-logistic models. In all cases, however, neither time nor any interactions with time

were statistically significant ($p > 0.05$), and thus it was determined that - given the impossibility of fitting a primary model and the clear indication that the overwhelming majority of inactivation was occurring, for all treatments and microbes, within the first 5 minutes - to refit the model using only the data from the 5min exposure.

4.2 Model Development and Evaluation

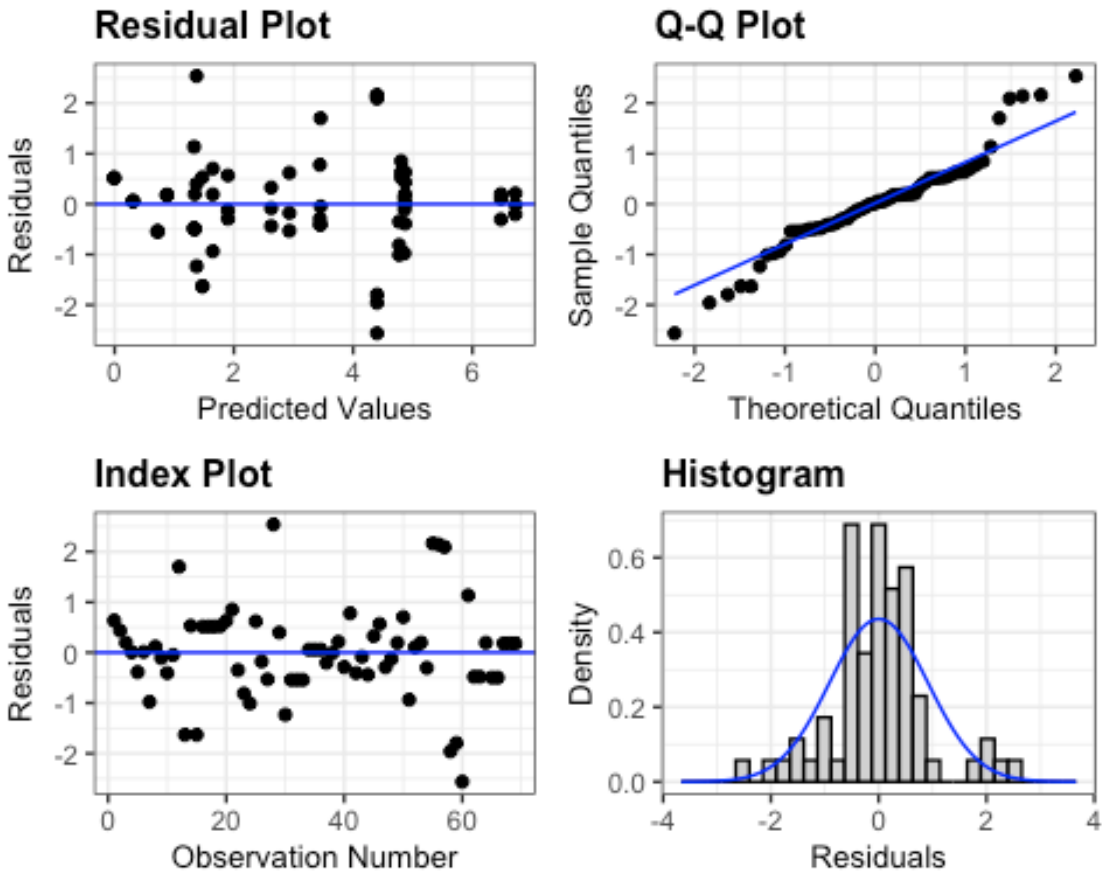
```
m1 = readRDS(here('study 1', 'Data (study 1)', 'obj1_m1.rda')) |>
  filter(sample != 'di' & time == 5) |>
  droplevels()

m1_mod_wb = gnls(survival ~ wb(x = chl.measured,
                               log10n0 = log10n0,
                               alpha = alpha,
                               beta = beta),
                 data = m1,
                 params = log10n0+alpha+beta ~ sample*microb,
                 start = list(log10n0 = c(6, rep(0, 3)),
                              alpha = c(5, rep(0, 3)),
                              beta = c(1, rep(0, 3))))

m1_mod_lg = gnls(survival ~ lg(x = chl.measured,
                               log10n0 = log10n0,
                               kappa = kappa,
                               sigma = sigma),
                 data = m1,
                 params = log10n0+kappa+sigma ~ sample*microb,
                 start = list(log10n0 = c(5, rep(0, 3)),
                              kappa = c(.1, rep(0, 3)),
                              sigma  = c(.2, rep(0, 3))))
```

Initial comparison via AICc indicates log-logistic is the best fitting:

```
compare_performance(m1_mod_wb, m1_mod_lg, metrics = c('AICc'))

# Comparison of Model Performance Indices

Name      | Model | AIC (weights) | AICc (weights) | BIC (weights) |    R2
-------------------------------------------------------------------------------
m1_mod_wb |  gnls | 158.8 (0.325) |  165.5 (0.325) | 187.9 (0.325) | 0.904
m1_mod_lg |  gnls | 157.4 (0.675) |  164.0 (0.675) | 186.4 (0.675) | 0.906

Name      |  RMSE | Sigma
-------------------------
m1_mod_wb | 0.634 | 0.697
m1_mod_lg | 0.627 | 0.690
```

Examination of the residuals suggests some evidence of heteroscedasticity (although likely actually the result of censoring of observations below LOD). Nevertheless, the model is compared with heteroscedasticity for any improvement.

```
m1_res_lg = resid(m1_mod_lg, type = 'normalized')
m1_fit_lg = fitted(m1_mod_lg)
resid_auxpanel(m1_res_lg, m1_fit_lg)
```



Power model is best fitting.

```
m1_mod_lg_exp = update(m1_mod_lg, weights = varExp())
m1_mod_lg_pow = update(m1_mod_lg, weights = varPower(),
                       control = gnlsControl(maxIter = 100))
m1_mod_lg_ident = update(m1_mod_lg, weights = varIdent(form = ~1|microb))

compare_performance(m1_mod_lg, m1_mod_lg_exp, m1_mod_lg_pow,
                    m1_mod_lg_ident, metrics = c('AICc')) |>
  dplyr::select(Name, Model, AICc, AICc_wt)

# Comparison of Model Performance Indices

Name            | Model | AICc (weights)
----------------------------------------
m1_mod_lg       |  gnls |  164.0 (0.130)
m1_mod_lg_exp   |  gnls |  165.9 (0.049)
```

```
m1_mod_lg_pow   |  gnls |  160.4 (0.768)
m1_mod_lg_ident |  gnls |  165.8 (0.053)
```

Refit the model using the re-parameterized version of the model. The code below converts the estimates in the original model into the corresponding estimates in the reparameterized model, so that they can be used as starting values.

```
# Extract design matrix for each treatment combination and name the rows
# accordingly
E = model.matrix(~ sample*microb, data = m1) |>
  unique()
ids = as.numeric(rownames(E))
rownames(E) = t(apply(m1[ids, c('sample', 'microb')], 1, paste0, collapse = '
'))

# Get ordering for marginal means and make sure the rows in the "E" matrix
match
#emmeans(m1_mod_lg, ~ sample:microb, param = 'kappa')
E = E[c(3, 4, 1, 2),]

# calculate marginal means for each treatment combination
kappa_emms = emmeans(m1_mod_lg_pow, ~ sample:microb, param = 'kappa')|>
    as.data.frame() |>
    dplyr::select(emmean) |>
    unlist()

n0_emms = emmeans(m1_mod_lg_pow, ~sample:microb, param = 'log10n0')|>
    as.data.frame() |>
    dplyr::select(emmean) |>
    unlist()

sigma_emms = emmeans(m1_mod_lg_pow, ~sample:microb, param = 'sigma') |>
    as.data.frame() |>
    dplyr::select(emmean) |>
    unlist()

# Calculate the time to 3 log10 reductions implied by the fitted estimates
# for each treatment combination
td_emms = log(10^3-1)*sigma_emms + kappa_emms

# Then, calculate the coefficients (based on the E matrix) implied by those
# 3 log10 reductions
td_coefs = round(solve(E)%*%td_emms, 1)
n0_coefs = round(solve(E)%*%n0_emms, 1)
sigma_coefs = round(solve(E)%*%sigma_emms, 1)

# These values should be extremely close to the fitted values in the
# reparameterized model
```

```
m1_mod = gnls(survival ~ lgrepar(x = chl.measured,
                                 log10n0 = log10n0,
                                 TD = TD,
                                 sigma = sigma,
                                 D = 3),
              data = m1,
              params = log10n0+TD+sigma ~ sample*microb,
              weights = varPower(),
              start = list(log10n0 = n0_coefs,
                           TD  = td_coefs,
                           sigma  = sigma_coefs))


saveRDS(m1_mod, here('study 1', 'Results (study 1)', "m1_mod.RDS"))

# Extract estimates and their 95% CI
m1_mod_coef = coef(m1_mod)
m1_mod_confint = confint(m1_mod)

as.data.frame (cbind(
  Coefs = names(m1_mod_coef),
  Est = round (as.numeric(m1_mod_coef), 3),
  lwr = round (m1_mod_confint[,1], 3),
  upr = round (m1_mod_confint[,2], 3)
  )) |>
 write_xlsx(here('study 1','Results (study 1)', "m1_mod_summary.xlsx"))

readxl::read_xlsx(here('study 1', 'Results (study 1)',
"m1_mod_summary.xlsx"))

# A tibble: 12 × 4
   Coefs                   Est    lwr    upr
   <chr>                   <chr>  <chr>  <chr>
 1 log10n0.(Intercept)     5.719  5.25   6.189
 2 log10n0.sample1         0.085  -0.385 0.555
 3 log10n0.microb1         0.879  0.409  1.349
 4 log10n0.sample1:microb1 0.036  -0.434 0.506
 5 TD.(Intercept)          8.547  6.455  10.638
 6 TD.sample1              3.36   1.269  5.452
 7 TD.microb1              -7.844 -9.936 -5.753
 8 TD.sample1:microb1      -3.462 -5.553 -1.371
 9 sigma.(Intercept)       0.603  0.538  0.668
10 sigma.sample1           -0.038 -0.103 0.027
11 sigma.microb1           0.136  0.071  0.201
12 sigma.sample1:microb1   0.089  0.024  0.154
```

4.3 Estimation, Inference, and Plotting

Only microbe species has a significant effect on the initial population (log10n0) parameter, while there is a significant interaction effect between microbe and sample on the chlorine

required for log 3 reduction (TD parameter), and for the sigma parameter (which characterizes the variation in chlorine susceptibility within the microbial population).

```
m1_mod = readRDS(here('study 1', 'Results (study 1)', "m1_mod.RDS"))

(m1_ftest_n0 = joint_tests(m1_mod, param = 'log10n0'))
```

```
 model term    df1 df2 F.ratio p.value
 sample          1  55   0.126  0.7245
 microb          1  55  13.443  0.0006
 sample:microb   1  55   0.022  0.8815
```

```
(m1_ftest_TD = joint_tests(m1_mod, param = 'TD'))
```

```
 model term    df1 df2 F.ratio p.value
 sample          1  55   9.917  0.0026
 microb          1  55  54.039  <.0001
 sample:microb   1  55  10.526  0.0020
```

```
(m1_ftest_sigma = joint_tests(m1_mod, param = 'sigma'))
```

```
 model term    df1 df2 F.ratio p.value
 sample          1  55   1.305  0.2583
 microb          1  55  16.783  0.0001
 sample:microb   1  55   7.204  0.0096
```

```
write_xlsx(list(log10n0 = as.data.frame(m1_ftest_n0),
                TD = as.data.frame(m1_ftest_TD),
                sigma = as.data.frame(m1_ftest_sigma)),
           path = here('study 1', 'Results (study 1)',
                       'm1_param_ftests.xlsx'))
```

Estimates and contrasts for the different parameters:***

```
(m1_emm_n0 = emmeans(m1_mod, pairwise ~ microb, param = 'log10n0'))
```

```
$emmeans
 microb emmean    SE df lower.CL upper.CL
 ec       6.60 0.413 55     5.77     7.43
 tv       4.84 0.243 55     4.35     5.33

Results are averaged over the levels of: sample
Confidence level used: 0.95

$contrasts
 contrast estimate    SE df t.ratio p.value
 ec - tv      1.76 0.479 55   3.666  0.0006

Results are averaged over the levels of: sample
```

```
(m1_emm_TD = emmeans(m1_mod, pairwise ~ microb:sample, param = 'TD'))
```

```
$emmeans
 microb sample emmean    SE df lower.CL upper.CL
 ec     ag1     0.601 0.648 55   -0.697     1.90
 tv     ag1    23.213 2.430 55   18.335    28.09
 ec     ag2     0.804 0.558 55   -0.314     1.92
 tv     ag2     9.568 3.400 55    2.754    16.38

Confidence level used: 0.95

$contrasts
 contrast        estimate    SE df t.ratio p.value
 ec ag1 - tv ag1  -22.612 2.520 55  -8.977  <.0001
 ec ag1 - ec ag2   -0.203 0.855 55  -0.238  0.9952
 ec ag1 - tv ag2   -8.967 3.460 55  -2.591  0.0575
 tv ag1 - ec ag2   22.409 2.500 55   8.973  <.0001
 tv ag1 - tv ag2   13.645 4.180 55   3.263  0.0100
 ec ag2 - tv ag2   -8.764 3.450 55  -2.544  0.0642

P value adjustment: tukey method for comparing a family of 4 estimates
```

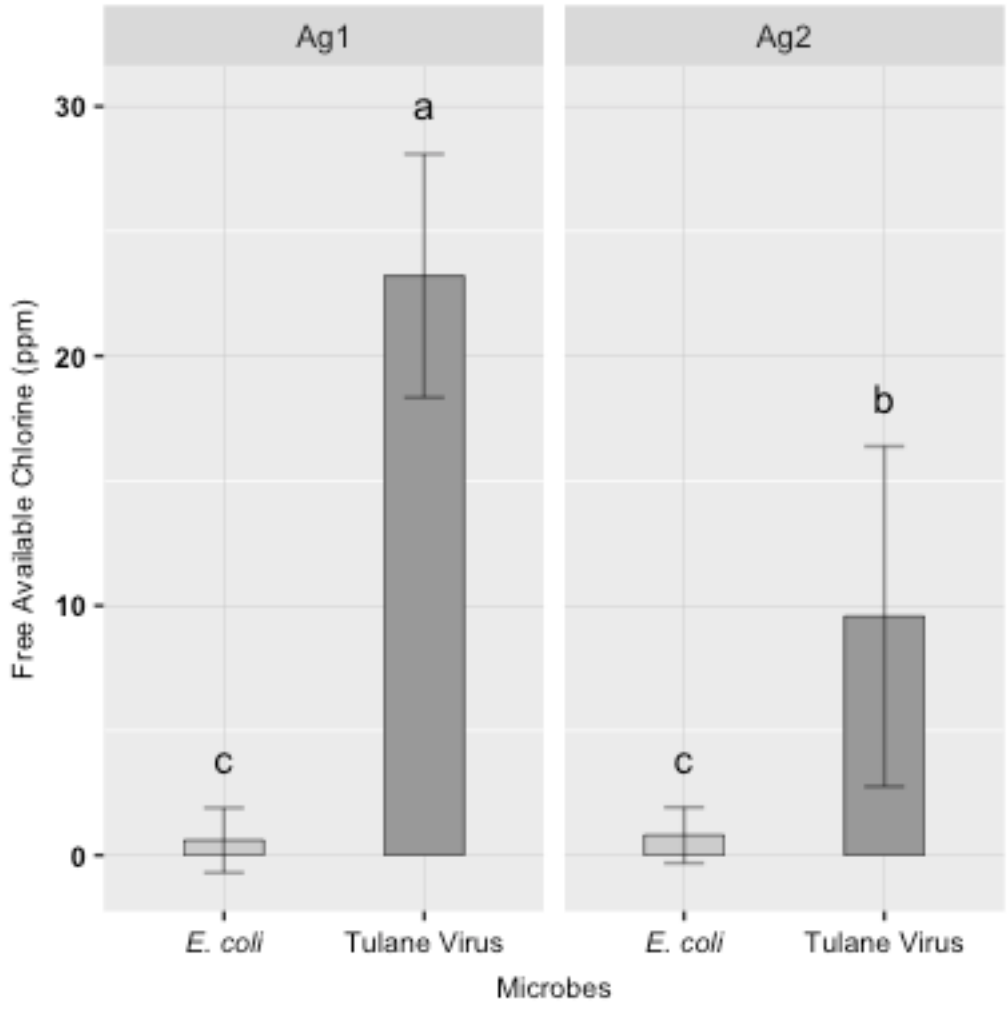
```
(m1_emm_sigma = emmeans(m1_mod, pairwise ~ microb:sample, param = 'sigma'))
```

```
$emmeans
 microb sample emmean     SE df lower.CL upper.CL
 ec     ag1     0.790 0.1140 55    0.561    1.019
 tv     ag1     0.340 0.0252 55    0.289    0.390
 ec     ag2     0.688 0.0413 55    0.605    0.770
 tv     ag2     0.594 0.0473 55    0.499    0.689

Confidence level used: 0.95

$contrasts
 contrast        estimate     SE df t.ratio p.value
 ec ag1 - tv ag1   0.4501 0.1170 55   3.849  0.0017
 ec ag1 - ec ag2   0.1023 0.1210 55   0.842  0.8339
 ec ag1 - tv ag2   0.1961 0.1240 55   1.586  0.3947
 tv ag1 - ec ag2  -0.3477 0.0484 55  -7.187  <.0001
 tv ag1 - tv ag2  -0.2540 0.0536 55  -4.740  0.0001
 ec ag2 - tv ag2   0.0938 0.0628 55   1.493  0.4487

P value adjustment: tukey method for comparing a family of 4 estimates
```

4.4 Plot 3-log10 reduction (TD)

```
m1_emm2_TD = emmeans(m1_mod, pairwise ~ sample:microb,
                     param = 'TD')

sample_labs = c(ag1 = 'Ag1', ag2 = 'Ag2')

m1_ch3d_plot = m1_emm2_TD |>
```

```
  cld(Letters = letters, reversed = T, adjust = 'none') |>
  as.data.frame() |>
  mutate(.group = gsub(' ', '', .group),
         Microbe = factor(microb, labels = c('E. coli', 'Tulane Virus')) ) |>
  ggplot(aes(x = Microbe, fill = Microbe))+
  facet_grid(cols = vars(sample), labeller = as_labeller(sample_labs))+
  geom_col(aes(y = emmean), show.legend = F, color = "black", linewidth =
0.2, width = 0.4) +
  geom_errorbar(aes(ymin = lower.CL, ymax = upper.CL), width = 0.2,
alpha=0.4)+
  geom_text(aes(y = upper.CL+2, label = .group)) +
  scale_y_continuous('Free Available Chlorine (ppm)')+
  scale_x_discrete('Microbes', labels = c(expression(italic("E. coli")),
"Tulane Virus"))+
  scale_fill_manual(values = c("E. coli" = "gray80", "Tulane Virus" =
"gray60")) +
 theme(
  plot.title = element_text(color="black", size=8, face = "bold", hjust =
0.5),
  axis.text.x=element_text(size = 8, color="black", face="bold", hjust = 0.5,
vjust = 0.5),
  axis.text.y=element_text (size = 8, color="black", face="bold"),
  axis.title.x = element_text (size = 8, vjust = -1),
  axis.title.y = element_text (size = 8, vjust = 2),
  panel.grid.major = element_line (color="gray", linewidth=0.1)
 )

m1_ch3d_plot
```

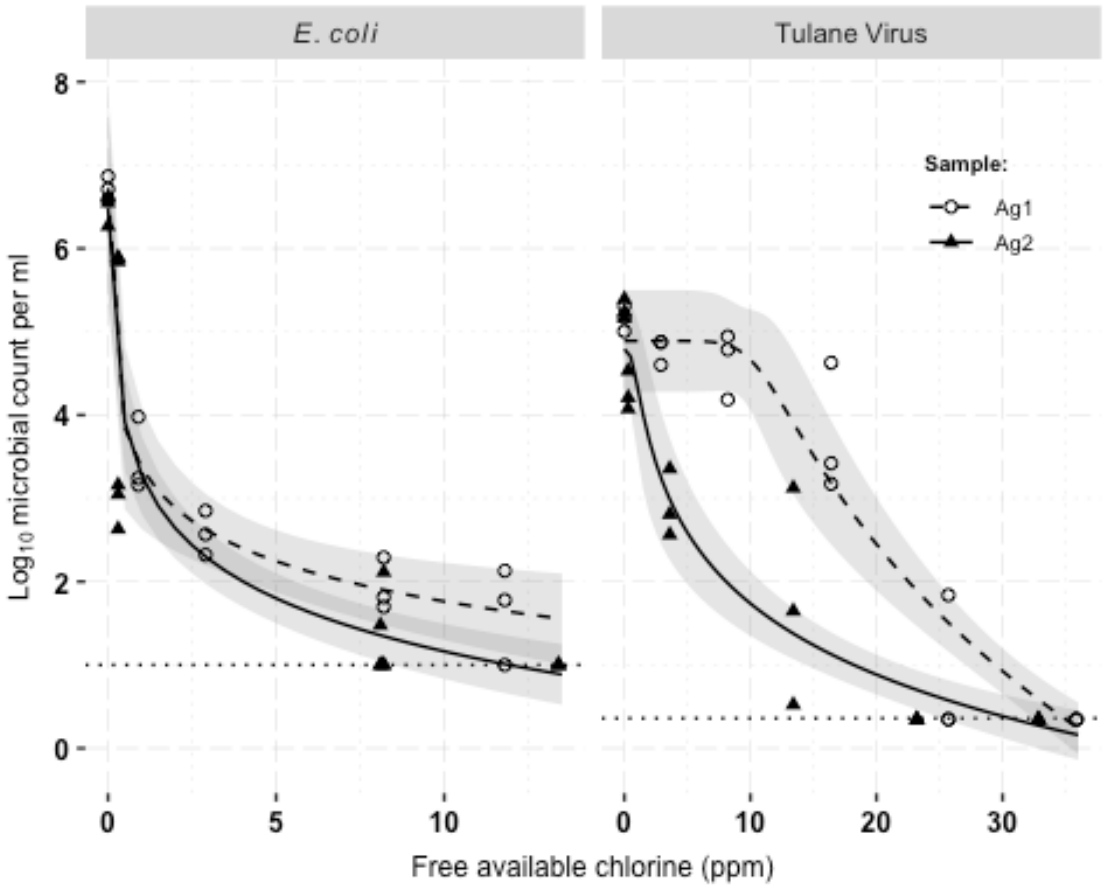



```
ggsave(plot=m1_ch3d_plot, file=here('study 1', 'Results (study 1)',
"m1_ch3d_plot.tiff"),
       width=14, height=7, units = c("cm"), dpi=600)
```

4.5 Plot microbial inactivation

```
m1_grid = expand.grid(sample = levels(m1$sample),
                      microb = levels(m1$microb),
                      chl.measured = seq(0, 36, 0.5)) |>
  filter(!(microb == 'ec' & chl.measured > 13.5))

m1_preds = avg_predictions(m1_mod, newdata = m1_grid,
                           by = c('sample', 'microb', 'chl.measured'))

microb_labs = c(ec = expression(italic('E. coli')), tv = 'Tulane Virus')

p_m1= m1_preds |>
  mutate(microb = factor(microb, labels = c('italic(E.~coli)',
```

```
'Tulane~Virus')),
         LOD = ifelse(microb == 'Tulane~Virus', 0.36, 1)) |>
  ggplot(aes(x = chl.measured)) +
  facet_grid(cols = vars(microb), labeller = label_parsed, scales = 'free_x')
+
  geom_hline(aes(yintercept = LOD), color = 'black', linetype = 3)+
  geom_ribbon(aes(ymin = conf.low, ymax = conf.high, group = sample), alpha =
.2,
              fill = 'grey45') +
  geom_line(aes(y = estimate, linetype = sample), color = "black") +
  geom_point(aes(y = survival, shape = sample),
             data = mutate(m1, microb = factor(microb,
                                               labels = c('italic(E.~coli)',
                                                          'Tulane~Virus'))),
             size = 1.5) +
  guides (fill = guide_legend(title = 'Sample:'),
          color = guide_legend(title = 'Sample:'),
          linetype = guide_legend(title = 'Sample:'),
          shape = guide_legend(title = 'Sample:')) +
  labs(x = "Free available chlorine (ppm)",
       y = bquote(Log[10]~microbial~count~per~ml)) +
  scale_fill_manual(labels = c('Ag1', 'Ag2')) +
  scale_shape_manual(values = c(1, 17),
                     labels = c('Ag1', 'Ag2')) +
  scale_linetype_manual(values = c("dashed", "solid"),
                        labels = c('Ag1', 'Ag2')) +
  theme(text = element_text(family = "Arial"),
        legend.position = 'inside', legend.position.inside = c(.88, .8),
        legend.direction = "vertical",
        legend.text = element_text(size = 7),
        legend.title = element_text(size = 7, face = "bold"),
        legend.key.height = unit(0.4, "cm"),
        plot.title = element_text(color = "black", size = 9, face = "bold",
hjust = 0.5),
        axis.text.x = element_text(size = 9, color = "black", face = "bold",
hjust = 0.5, vjust = 0.5),
        axis.text.y = element_text(size = 9, color = "black", face = "bold"),
        axis.title.x = element_text(size = 9, vjust = -1),
        axis.title.y = element_text(size = 9, vjust = 2),
        panel.grid.major = element_line(
          color = "gray", linewidth = 0.1, linetype = "longdash"),
        panel.grid.minor = element_line(
          color = "lightgray", linewidth = 0.1, linetype = "dotted"),
        panel.background = element_rect(fill = "transparent"),
        legend.background = element_rect(fill = "transparent", color =
"transparent"),
        plot.background = element_rect(fill = "transparent", color =
"transparent"))

p_m1
```




```
ggsave(file = here('study 1', 'Results (study 1)', "Fig_3_trend_plot.tiff"),
       plot = p_m1,
       width=14, height=7, units = c("cm"), dpi=600)
```

These are model-based estimates of mean (along with their standard error) of the log10 PFU for each treatment combination.

```
m1_tab_grid = m1 |>
  dplyr::select(sample, microb, chl.added) |>
  rename(chl.measured = chl.added) |>
  distinct()
m1_ests = avg_predictions(m1_mod, newdata = m1_tab_grid,
                          by = c('microb', 'sample', 'chl.measured'))
m1_ests_tab = m1_ests |>
  as.data.frame() |>
  rename(Sample = sample, Microbe = microb) |>
  mutate(across(4:5, as.numeric),
         estimate = paste0(sprintf("%.2f", estimate),
                           ' \u00b1 ', sprintf("%.2f", std.error)),
         chl.measured = paste0(chl.measured, ' ppm')) |>
  dplyr::select(Microbe, Sample, chl.measured, estimate) |>
  pivot_wider(names_from = chl.measured, values_from = estimate,
```

```
              names_prefix = '')
print(m1_ests_tab)
```

```
# A tibble: 4 × 10
  Microbe Sample `0 ppm`     `2 ppm`     `4 ppm`     `10 ppm`    `15 ppm`
  <fct>   <fct>  <chr>       <chr>       <chr>       <chr>       <chr>
1 ec      ag1    6.72 ± 0.59 2.88 ± 0.26 2.40 ± 0.19 1.76 ± 0.24 1.48 ± 0.29
2 ec      ag2    6.48 ± 0.58 2.64 ± 0.19 2.00 ± 0.16 1.16 ± 0.17 0.79 ± 0.19
3 tv      ag1    4.89 ± 0.31 <NA>        4.89 ± 0.31 4.66 ± 0.31 <NA>
4 tv      ag2    4.79 ± 0.37 <NA>        2.86 ± 0.35 1.74 ± 0.20 <NA>
  `20 ppm`    `30 ppm`    `40 ppm`
  <chr>       <chr>       <chr>
1 <NA>        <NA>        <NA>
2 0.53 ± 0.21 <NA>        <NA>
3 2.45 ± 0.28 0.93 ± 0.14 -0.16 ± 0.19
4 0.88 ± 0.12 0.38 ± 0.13 0.03 ± 0.17
```

```
write_xlsx(m1_ests_tab,
           path = here('study 1', 'Results (study 1)',
                       'Estimated Conc. for 3 Log Reduction.xlsx'))
```

5. Reparameterizing the Log-Logistic Model

$$log_{10}(N_t) = log_{10}(N_0) - log_{10}\left(1 + e^{\frac{log(x)-\kappa}{\sigma^2}}\right)$$

Letting C_3 be the concentration at which $log_{10}(N_t)$ is 3 below $log_{10}(N_0)$, it follows that:

$$log_{10}(N_0) - 3 = log_{10}(N_0) - log_{10}\left(1 + e^{\frac{log(C_3)-\kappa}{\sigma^2}}\right)$$

Subtracting $log_{10}(N_0)$ and multiplying by -1 gives:

$$log_{10}\left(1 + e^{\frac{log(C_3)-\kappa}{\sigma^2}}\right) = 3$$

$$1 + e^{\frac{log(C_3)-\kappa}{\sigma^2}} = 10^3$$

$$e^{\frac{log(C_3)-\kappa}{\sigma^2}} = 10^3 - 1$$

$$\frac{log(C_3) - \kappa}{\sigma^2} = log(10^3 - 1)$$

$$log(C_3) - \kappa = log(10^3 - 1)\sigma^2$$

$$\kappa = log(C_3) - log(10^3 - 1)\sigma^2$$

And substituting this back into the original form of the model gives:

$$log_{10}(N_t) = log_{10}(N_0) - log_{10}\left(1 + e^{\frac{log(x) - \left(log(C_3) - log\left(10^3 - 1\right)\sigma^2\right)}{\sigma^2}}\right)$$